

Министерство образования Республики Беларусь

Учреждение образования
**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ**

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Проектирование защищённых интеллектуальных
информационных систем»
на тему:

Управление доступом с помощью списков контроля доступа.

**Часть 1. Управление доступом к объектам операционных
систем.**

**Часть 2. Управление доступом к приложениям и системам
базам данных**

Студенты гр. 321701

И. А. Политыко

В. А. Лукашов

Руководитель

Д. А. Сальников

Минск 2026

СОДЕРЖАНИЕ

Перечень условных обозначений	3
1 Постановка задачи	4
2 Часть 1. Управление доступом к объектам операционных систем	5
3 Часть 2. Управление доступом к приложениям и системам ба- зам данных	8
Заключение	9
Список использованных источников	10

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

ACL	— Access Control List, Список контроля доступа;
ACE	— Access Control Entry, Запись списка контроля доступа;
DAC	— Discretionary Access Control, Избирательное управление доступом;
ОС	— Операционная система;
СУБД	— Система управления базами данных;
БД	— База данных;
SQL	— Structured Query Language, Язык структурированных запросов;
DDL	— Data Definition Language, Язык определения данных;
DML	— Data Manipulation Language, Язык манипулирования данными;
CRUD	— Create, Read, Update, Delete, Базовые операции над данными;
GRANT / REVOKE	— Команды назначения и отзыва привилегий в СУБД;
rwX	— Права чтения (read), записи (write) и исполнения (execute);
UID / GID	— Идентификаторы пользователя и группы в UNIX-подобных ОС.

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы является изучение механизмов избирательного управления доступом: на уровне объектов файловой системы операционной системы Linux и на уровне ролей и привилегий в системе управления базами данных PostgreSQL.

В части 1 требуется разработать программу (набор сценариев командной оболочки), которая создаёт группы пользователей `group_iit1`, `group_iit2` и пользователей `iit11`, `iit12`, `iit21`, `iit22`, `iit3` с указанным членством в группах, а также предоставляет пользователю `iit21` административные привилегии; создаёт каталог `/home/pzs` и подкаталоги `pzs11–pzs15` с правами доступа только для владельца, только для группы, только для остальных, для всех пользователей и только для администратора (`root`); от имени пользователя `iit11` создаёт в доступных каталогах набор файлов `file11–file55` с комбинациями прав чтения, записи и исполнения; проверяет для указанных пользователей и суперпользователя возможность чтения, изменения и исполнения файлов, остановки процессов, запущенных по шаблону `filex5`, а также чтения содержимого каталогов, создания и удаления файлов в них; удаляет созданные файлы, каталоги, пользователей и группы.

В части 2 (вариант — PostgreSQL) требуется установить СУБД, настроить механизм контроля доступа с ролями администратора (полный доступ), пользователя (CRUD) и гостя (только чтение), проверить корректность политики безопасности и удалить созданные объекты СУБД.

2 ЧАСТЬ 1. УПРАВЛЕНИЕ ДОСТУПОМ К ОБЪЕКТАМ ОПЕРАЦИОННЫХ СИСТЕМ

Краткое описание программы. Часть 1 реализована набором сценариев командной оболочки. Сценарий настройки создаёт группы, пользователей, каталоги и файлы по пунктам 1–13 методички. Три сценария проверки охватывают чтение, запись и исполнение файлов, остановку процессов шаблона `filex5` и операции в каталогах. Отдельный сценарий удаляет стенд.

В модели DAC UNIX-подобных систем права владельца проверяются раньше прав группы. Поэтому каталог с режимом «только для группы» (070) нельзя отдавать пользователю `iit11` как владельцу: иначе у него окажутся нулевые биты владельца, и доступ по группе не сработает. Аналогично для режима «только остальные» (007). Принятая схема владельцев каталогов приведена в таблице 2.1.

Каталог	Режим	Владелец:группа	Смысл
pzs11	700	iit11:group_iit1	только владелец
pzs12	070	root:group_iit1	только группа
pzs13	007	root:root	только остальные
pzs14	777	iit11:group_iit1	все пользователи
pzs15	700	root:root	только администратор

Таблица 2.1 — Каталоги `/home/pzs` и права доступа

Содержимое файлов соответствует методичке: для шаблона `filex5` (`file15`, `file25`, ..., `file55`) — строка `read testVariable`; для остальных — `echo "Hello World"`. Файлы `file51`–`file55` передаются администратору (`chown root:root`). Проверки доступа неразрушающие для содержимого: запись в файл выполняется с откатом; в каталогах сначала создаётся зонд, затем проверяется удаление каждого существующего `file*` с восстановлением из копии. Исполнение — фактический запуск с таймаутом, а не проверка бита `test -x`: скрипту с `#!/bin/bash` недостаточно бита `x`, интерпретатору нужно ещё и чтение. Поэтому файл режима «только исполнение» (100, 111) при запуске даёт отказ. Каталог стенда — `/home/pzs`: методичка путь не фиксирует, выбран каталог вне домашних каталогов учебных учёток.

Проверка политики. Порядок: сначала сценарий настройки от суперпользователя, затем три сценария проверки, в конце — очистка. Результаты пишутся в текстовые таблицы. Экспериментальный прогон выполнен 10.09.2026 на хосте с ОС Linux.

Фрагмент проверки каталогов (п. 16) приведён в таблице 2.2 (значение «ДА» означает успешные `list`, `create` и удаление существующих `file*` с

восстановлением). Для удаления важен режим каталога, а не биты самого файла: без флага `sticky` достаточно записи в каталог.

Пользователь	pzs11	pzs12	pzs13	pzs14	pzs15	
iit11	ДА	ДА	ДА	ДА	НЕТ	
iit12	НЕТ	ДА	ДА	ДА	НЕТ	
iit21	НЕТ	НЕТ	ДА	ДА	НЕТ	
iit22	НЕТ	НЕТ	ДА	ДА	НЕТ	
iit3	НЕТ	НЕТ	ДА	ДА	НЕТ	
root	ДА	ДА	ДА	ДА	ДА	

Таблица 2.2— Доступ к каталогам (*list / create / delete*), 10.09.2026

Результаты п. 15 (остановка процессов с телом `filex5`, запущенных от `iit11`) приведены в таблице 2.3. Оригиналы `file15`–`file55` имеют режимы без чтения для интерпретатора, поэтому процесс стартует обёрткой с тем же текстом и битами `r+x`: проверяется право `kill`, а не повторный разбор `shebang`. Учётка `iit21` входит в `sudo`, но проверка идёт без повышения прав, поэтому `kill` чужого процесса ей недоступен — как обычному пользователю.

Файл	iit11	iit12	iit21	iit22	iit3	root	
file15	ДА	НЕТ	НЕТ	НЕТ	НЕТ	ДА	
file25	ДА	НЕТ	НЕТ	НЕТ	НЕТ	ДА	
file35	ДА	НЕТ	НЕТ	НЕТ	НЕТ	ДА	
file45	ДА	НЕТ	НЕТ	НЕТ	НЕТ	ДА	
file55	ДА	НЕТ	НЕТ	НЕТ	НЕТ	ДА	

Таблица 2.3— Возможность *kill* процесса *filex5* (п. 15)

Фрагмент проверки файлов (п. 14) для пользователя `iit11` в каталоге `pzs11` — таблица 2.4.

Ключевые фрагменты сценария настройки приведены в листингах 2.1 и 2.2.

```

1 chown iit11:group_iit1 "${BASE}/pzs11"; chmod 700 "${BASE}/pzs11"
2 chown root:group_iit1  "${BASE}/pzs12"; chmod 070 "${BASE}/pzs12"
3 chown root:root        "${BASE}/pzs13"; chmod 007 "${BASE}/pzs13"
4 chown iit11:group_iit1 "${BASE}/pzs14"; chmod 777 "${BASE}/pzs14"
5 chown root:root        "${BASE}/pzs15"; chmod 700 "${BASE}/pzs15"

```

Листинг 2.1— Владельцы и режимы каталогов

```

1 for d in pzs11 pzs12 pzs13 pzs14; do
2   runuser -u iit11 -- "${CREATE_HELPER}" "${BASE}/${d}"
3 done

```

Файл	R	W	X	Комментарий
file11	ДА	НЕТ	НЕТ	только чтение владельца
file12	ДА	ДА	НЕТ	чтение и запись владельца
file13	НЕТ	ДА	НЕТ	только запись владельца
file14	ДА	ДА	ДА	rwX владельца
file15	НЕТ	НЕТ	НЕТ	бит x без чтения; shebang не запускается
file21–file25	НЕТ	НЕТ	НЕТ	владелец; биты owner=0
file41	ДА	НЕТ	НЕТ	права «для всех»
file44	ДА	ДА	ДА	права «для всех»
file45	НЕТ	НЕТ	НЕТ	111: бит x без чтения
file51–file55	НЕТ	НЕТ	НЕТ	владелец root

Таблица 2.4— Доступ *iit11* к файлам в *pzs11* (*READ* / *WRITE* / *EXEC*)

```

4 # file51..file55 -> root
5 for d in pzs11 pzs12 pzs13 pzs14; do
6     chown root:root "${BASE}/${d}/file5"{1,2,3,4,5}
7 done

```

Листинг 2.2— Создание файлов от имени *iit11*

3 ЧАСТЬ 2. УПРАВЛЕНИЕ ДОСТУПОМ К ПРИЛОЖЕНИЯМ И СИСТЕМАМ БАЗАМ ДАННЫХ

Краткое описание выбранной СУБД. PostgreSQL — объектно-реляционная система управления базами данных с моделью ролей и привилегий (GRANT/REVOKE) на уровне баз, схем, таблиц, последовательностей и функций. Роли могут наследовать права других ролей, что позволяет отделить политику доступа от учёток входа.

Описание программ настройки политики. Сценарий установки поднимает пакеты и службу. Сценарий политики создаёт базу, схему, три роли и таблицу, настраивает аутентификацию. Сценарий проверки строит матрицу SELECT / INSERT / UPDATE / DELETE / CREATE / DROP. Сценарий удаления снимает объекты стенда.

В системе три роли по методичке (таблица 3.1). Объекты: база lab_db, схема lab_schema, таблица lab_schema.test_table.

Учётка	Роль	Права
admin_user	admin_role	полный доступ к схеме (DML и DDL)
app_user	user_role	SELECT, INSERT, UPDATE, DELETE
guest_user	guest_role	только SELECT

Таблица 3.1— Роли и учётные записи PostgreSQL

Прогон выполнен 10.09.2026. Проверка политики завершилась без расхождений с ожиданиями. Сводка — таблица 3.2.

Учётка	SELECT	INSERT	UPDATE	DELETE	CREATE	DROP	
admin_user	ДА	ДА	ДА	ДА	ДА	ДА	
app_user	ДА	ДА	ДА	ДА	НЕТ	НЕТ	
guest_user	ДА	НЕТ	НЕТ	НЕТ	НЕТ	НЕТ	

Таблица 3.2— Результаты проверки ACL PostgreSQL, 10.09.2026

Таким образом, администратор имеет полный доступ, пользователь приложения — только DML (CRUD), гость — только чтение, что соответствует требованиям методички. Фрагмент назначения привилегий приведён в листинге 3.1.

```
1 GRANT ALL PRIVILEGES ON SCHEMA lab_schema TO admin_role;
2 GRANT CREATE ON SCHEMA lab_schema TO admin_role;
3 GRANT SELECT, INSERT, UPDATE, DELETE
4   ON ALL TABLES IN SCHEMA lab_schema TO user_role;
5 GRANT SELECT ON ALL TABLES IN SCHEMA lab_schema TO guest_role;
```

Листинг 3.1— Назначение привилегий ролям

ЗАКЛЮЧЕНИЕ

В рамках лабораторной работы №1 исследованы и программно реализованы механизмы избирательного управления доступом на двух уровнях: объектов файловой системы Linux и ролевой модели PostgreSQL. Экспериментальный прогон выполнен 10.09.2026.

Семантическая составляющая. Работа демонстрирует принципы ACL/DAC: в операционной системе — через классы «владелец / группа / остальные» и биты `rwX`, в СУБД — через роли и привилегии SQL. Показано, что эффективные права зависят не только от заданного набора битов или `GRANT`, но и от правил проверки доступа (в UNIX владелец не «проваливается» в групповые права) и от наследования ролей PostgreSQL.

Прагматическая составляющая. Подготовлен воспроизводимый стенд: автоматическое развёртывание, неразрушающая проверка политики и контролируемая очистка. Такой подход снижает риск ошибок ручной настройки и позволяет многократно демонстрировать и проверять ACL перед сдачей лабораторной.

Поставленные цели достигнуты: программы настройки и проверки доступа работоспособны; для PostgreSQL матрица операций дала `FAILS=0` (администратор — полный доступ, пользователь — CRUD без DDL, гость — только `SELECT`).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Олифер В. Г., Олифер Н. А. Сетевые операционные системы: учебник для вузов. — 2-е изд. — СПб. : Питер, 2009. — 669 с.
- [2] Таненбаум Э. С. Современные операционные системы. — 3-е изд. — СПб. : Питер, 2011. — 1120 с.
- [3] Ховард М., Лебланк Д. Защищённый код. — М. : Русская редакция, 2004. — 704 с.
- [4] CWE: Common Weakness Enumeration [Электронный ресурс]. — Режим доступа: <http://cwe.mitre.org/>. — Дата доступа: 10.09.2026.
- [5] PostgreSQL Documentation [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/>. — Дата доступа: 10.09.2026.
- [6] chmod(1) — Linux manual page [Электронный ресурс]. — Режим доступа: <https://man7.org/linux/man-pages/man1/chmod.1.html>. — Дата доступа: 10.09.2026.